机器学习LAB2报告

522031910213 朱涵

1 系统实现

对原代码进行改造，采用deepseek-v3模型，采用BERT-BASE-UNCASED预训练模型用于文本的向量嵌入。

具体步骤包括加载知识库，切片预处理，向量嵌入。

其中chunksize为256个字符，切片处理后采用BERT模型的最后一层隐藏层的状态作为嵌入向量，保存到pkl文件中方便后续复用。

核心代码如下图，使用BERT模型将文本嵌入向量。

```python
def get_bert_embedding(text: str) -> np.ndarray:
    """Get BERT embedding for a piece of text."""
    inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=512).to(device)
    with torch.no_grad():
        outputs = bert_model(**inputs)
    # Use mean pooling of the last hidden states
    embeddings = outputs.last_hidden_state.mean(dim=1).squeeze().cpu().numpy()
    return embeddings
```

2 评估结果

进行向量嵌入后，可以对知识库中的问题开始进行测试。检索采用top-10，相似度依据是余弦相似度。检索后合并上下文和问题发送给LLM，并且提供适当的提示词对模型的输出进行格式约束。将LLM的回答保存下来，和标准答案一一进行对比，计算出评分。

关键代码如下图，用于检索和问题相关的chunk。

```python
def retrieve_relevant_chunks(question: str, embeddings: np.ndarray, chunks: List[str],top_k: int = TOP_K) -> List[str]:
    """Retrieve top-k most relevant chunks for a question."""
    question_embedding = get_bert_embedding(question).reshape(1, -1)
    similarities = cosine_similarity(question_embedding, embeddings)[0]
    top_indices = np.argsort(similarities)[-top_k:][::-1]
    print(f"Top {top_k} indices: {top_indices}")
    return [chunks[i] for i in top_indices]
```

最终评分如下图。

```
==============================
Overall Scores:
==============================
Average EM: 0.2450 (24.50%)
Average F1: 0.3490 (34.90%)
==============================
```

3 结果分析

最终评分不高的原因可能有2点：

- LLM本身性能较差；
- 向量检索的质量较差，提供的上下文和问题不符；

本人尝试将几个未答对的问题和对应上下文直接提供给LLM进行回答，发现基本上可以得到正确答案，说明第一点不是评分不高的原因，接下来我将给出一些可以提高RAG质量的方法，并尝试改进代码。

想要提高RAG的质量，有几种解决方案：

- 替换向量嵌入模型为更优秀的模型，比如BGE；
- 将问题分解为多个子问题，从而检索出更加相关的信息；
- 为LLM提供更多或更完整的chunk，从而使得上下文覆盖到问题的概率增加；

对于第一点，由于BERT本身已经是本地设备可以运行的比较好的模型，本人不打算采用更复杂的模型进行嵌入。

对于第二点，本人将在后面给出改进方案和结果。

对于第三点，可以预见如果提高chunksize或topK参数，LLM必定会获取到更加完善的上下文，但一方面LLM的输入token数有所限制，另一方面给出更多的上下文是治标不治本的方法，以高性能损耗换取了较低的效益，不值得尝试。

然而，本人联想到平时使用LLM的经验，一般我们都是给出完整的上下文信息，而在进行chunk划分后，RAG提供的上下文往往是不连续甚至不属于同一个context的文本，这些文本简单拼接在一起可能会丧失一些语义，甚至影响LLM的判断。因此本人尝试建立chunk到context的反向索引，在提供上下文时只提供一个完整的context给LLM，最终发现效果较好。具体方案可见下一节。

4 策略改进

4.1 提供完整上下文

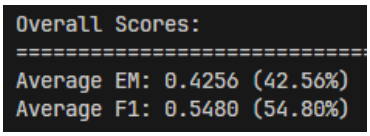
如上一节所说，我们需要提供完整的context给LLM，使得LLM获取到的语义是完整且独立的。

关键代码如下图，建立chunk到完整context的反向索引，在检索时，取出chunk属于的context组合成上下文提供给LLM。

```python
def retrieve_relevant_chunks(question: str, embeddings: np.ndarray, chunks: List[str],chunk_to_context:dict, top_k: int = TOP_K) -> List[str]:
    """Retrieve top-k most relevant chunks for a question."""
    question_embedding = get_bert_embedding(question).reshape(1, -1)
    similarities = cosine_similarity(question_embedding, embeddings)[0]
    top_indices = np.argsort(similarities)[-top_k:][::-1]
    print(f"Top {top_k} indices: {top_indices}")
    # 获取top-k chunks对应的contexts
    retrieved_contexts = []
    seen_contexts = set()  # 避免重复context
    for i in top_indices:
        chunk = chunks[i]
        context = chunk_to_context.get(chunk)
        if context and context not in seen_contexts:
            retrieved_contexts.append(context)
            seen_contexts.add(context)
            if len(retrieved_contexts) >= top_k:
                break
    print(f"Retrieved {len(retrieved_contexts)} relevant contexts")

    return retrieved_contexts
```

最终评分如下图。



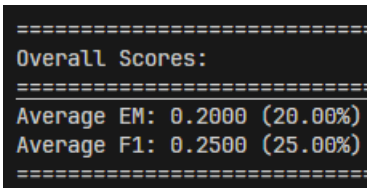
从output文件中可以看出，这个准确率基本上就是向量相关度检索的准确率，也就是说，只要能够检索到对应问题的chunk，LLM就会给出正确的回答，反之一旦搜索不到对应的上下文，LLM只能给出“缺少上下文，无法回答”的回复。这个方法的缺点是给出的完整context可能会超出token限制，或者由于长度而影响响应的性能。

4.2 子问题分解

如文档所说，一般的多跳检索需要对问题进行分解，使得向量的检索效果更好，且提供的上下文更完整。

大致思路为：先使用LLM做子问题的分解，对子问题进行向量检索，将检索结果作为上下文提供给LLM，得到子问题的答案，最终聚合子问题的答案作为原问题的上下文提供给LLM，得到最终答案。具体代码较长，在此不作展示。

由于token限制，最终仅在前10个问题上进行测试，评分如下图。



可以看到评分较低，甚至不如无RAG系统的评分。推测可能是问题分解的质量较差，查看详细的子问题内容，发现大多数子问题分解的都不符合原问题。甚至如第一个问题，问题分解后直接返回了3个原问题。一方面可能是使用LLM做问题分解的质量不好，另一方面可能是数据集中的问题都比较简单，不太适合进一步分解。

5 总结

可以看出RAG的关键是向量检索的质量，一旦检索到对应的上下文，LLM基本就能给出正确的回答，反之则很难给出正确回答。提升检索质量的方法有很多，本人列出的几点都可以实现，后续可以参阅其他资料，给出更加成熟完善的方案实现多跳RAG系统。